CAHIER DE MISE EN OEUVRE

THEME: CONCEPTION ET REALISATION DE BOOT INSPECTOR, UN OUTIL D'ANALYSE DU DEMARRAGE D'UNE DISTRIBUTION LINUX: Cas de Manjaro.

Rédigé par:

- TCHINDA METEWOUE Florelle Ecladore
- OMGBA ESSENGUE Patrice

ENCADRANT: M. NGUIMBUS Emmanuel

Master 1 Sécurité des Systèmes d'Information 2025/2026

PLAN

1. INTRODUCTION
2. ANALYSE DE L'EXISTANT
3. ARCHITECTURE DE LA SOLUTION
4. CHOIX TECHNOLOGIQUES ET JUSTIFICATION
5. ENVIRONNEMENT DE DEVELOPPEMENT
6. IMPLEMENTATION TECHNIQUE
7. Conclusion

INTRODUCTION

Contexte général

Les systèmes d'exploitation modernes reposent sur des mécanismes complexes de gestion du démarrage. Sous Linux, ce processus est orchestré par systemd, qui initialise le noyau, lance les services système et prépare l'environnement utilisateur.

Toutefois, l'analyse des performances de démarrage demeure essentiellement textuelle et technique. Les outils natifs tels que *systemd-analyze* ou *journalctl* produisent des sorties détaillées mais peu accessibles aux utilisateurs non experts.

INTRODUCTION

Problématique

Comment faciliter l'analyse des performances de démarrage d'un système Linux en transformant des données techniques textuelles en informations visuelles exploitables et compréhensibles ?

Les limites identifiées sont :

- ✓ Complexité des logs système;
- ✓ Difficulté d'identification rapide des services ralentissant le démarrage;
- ✓ Absence d'interface graphique synthétique.

INTRODUCTION

Objectifs du projet

Le projet Boot Inspector vise à :

- ✓ Collecter les données de démarrage via les outils système natifs;
- ✓ Traiter et structurer ces données;
- ✓ Les présenter sous forme de tableau de bord interactif;
- ✓ Permettre l'identification rapide des services les plus lents;
- ✓ Générer un rapport exportable (PDF).

FR

ANALYSE DE L'EXISTANT

Outils natifs disponibles du système

Les principales commandes utilisées sous Linux sont :

- ✓ systemd-analyze
- ✓ systemd-analyze blame
- ✓ journalctl -b

Ces outils permettent respectivement **de mesurer le temps total de démarrage, d'identifier les services les plus lents et d'examiner les journaux système du dernier boot**

FR

ANALYSE DE L'EXISTANT

Limites des outils natifs disponibles

Malgré leur précision, ces outils présentent plusieurs limites :

- ✓ Interface exclusivement textuelle
- ✓ Difficulté d'interprétation rapide
- ✓ Absence de visualisation graphique
- ✓ Courbe d'apprentissage élevée pour les utilisateurs novices

Ces constats justifient le développement d'un outil intermédiaire facilitant l'analyse.

FR

ARCHITECTURE DE LA SOLUTION

Architecture générale

Boot Inspector repose sur une architecture client-serveur intégrée organisée en 03 couches principales :

- Présentation (Frontend)
- Accès Système (Backend)
- Persistance (Base de données)

FR

A. Couche Présentation (Frontend)

- Technologies :
- • React 19
- • Tailwind CSS
- • Lucide Icons

- Rôle :
- • Transformer les données brutes en graphiques et tableaux lisibles.

- Gestion d’état :
- • AppProvider.tsx gère le thème, la langue et l’état système.

FR

B. Couche Accès Système (Backend)

- Fichier clé : src/app/actions/system.ts
- Rôle :
- • Tunnel sécurisé entre navigateur et système Linux.
- • Exécution des commandes via Node.js local.
- • Communication avec le noyau Linux.

✓Interaction avec le système

Le backend exécute des commandes système à l'aide de la fonction *exec* de Node.js afin de :

- Capturer les temps de démarrage
- Identifier les services critiques

C. Couche Persistance

- Technologie : Google Cloud Firestore (NoSQL)

- Rôle :
- • Stockage des analyses terminées.
- • Suivi de l’évolution des performances sur plusieurs semaines.

CHOIX TECHNOLOGIQUE ET JUSTIFICATION

FR

Choix du framework: Next.js

Next.js est la structure principale de l'application qui permet une architecture full-stack unifiée, qui intègre les Server Actions pour séparer clairement client et serveur et qui facilite le développement et le déploiement.

Bibliothèque d'interface: React

React est utilisé pour créer les composants interactifs (boutons, graphiques, tableaux) offrant une architecture basée sur des composants réutilisables, une mise à jour dynamique de l'interface et une bonne maintenabilité du code.

Framework CSS: Tailwind CSS

Tailwind CSS permet de créer un design moderne et "propre" sans écrire de fichiers CSS complexes pour un développement rapide d'interfaces modernes et une approche utilitaire simplifiant la maintenance.

Lucide React:

Pour les icônes illustrant les différentes fonctions.

ENVIRONNEMENT DE DEVELOPPEMENT

FR

Environnement materiel

- ✓ Hyperviseur: Oracle VirtualBox;
- ✓ Machine virtuelle sous Manjaro Arch Linux;
- ✓ Processeur 2 GHz;
- ✓ Mémoire vive ≥ 4 Go.

ENVIRONNEMENT DE DEVELOPPEMENT

FR

Environnement logiciel

- ✓ Distribution Linux (Manjaro Arch Linux) utilisant systemd;
- ✓ Node.js (version récente stable);
- ✓ Gestionnaire de paquets npm;
- ✓ Navigateur web moderne.

IMPLEMENTATION TECHNIQUE

FR

Structure du projet

Le projet est organisé comme suit :

- /app : interface principale;
- /actions : logique serveur;
- /components : composants React;
- /public : ressources statiques.

FR

IMPLEMENTATION TECHNIQUE

Mécanismes d’exécution des commandes

Lorsqu’un utilisateur déclenche l’analyse :

1) **Requête:** Le frontend (navigateur) envoie une requête au serveur local de l’application;

2) **Exécution:** Le serveur exécute une commande **Node.js** nommée *exec*. C'est comme si l'application ouvrait un terminal invisible et tapait *systemd-analyze*;

3) **Capture:** Le serveur récupère le texte renvoyé par le terminal;

4) **Traduction:** Le texte brut est envoyé à l'interface **React** et les données sont transformées en format exploitable;

5) **Affichage:** L’interface affiche les résultats en graphiques et en badges colorés.

FR

IMPLEMENTATION TECHNIQUE

Sécurisation

- ✓ Les commandes exécutées sont prédéfinies.
- ✓ Aucune entrée utilisateur n’est injectée dans la commande.
- ✓ L’application fonctionne en lecture seule.
- ✓ Aucun fichier système n’est modifié.

CONCLUSION

Le projet Boot Inspector répond à une problématique concrète liée à la difficulté d'analyse des performances de démarrage sous Linux.

En proposant une interface graphique interactive exploitant les données système natives, l'application permet une compréhension rapide et intuitive des phases de démarrage et des services impactant les performances. Ce pendant nous tenons a noter nos difficulté concernant la fonctionnalité principale c'est-à-dire l'échange entre notre serveur application et le système linux ou il est actif.